FinanceOS - Personal Finance Operating System

A private, **fully offline** Windows desktop application that helps you track every expense, manage debts, hit savings goals, and see your full financial picture in one place.

Built for one person - you. No accounts to create, no cloud, no syncing. Just open the app on your PC and your data is there.

Why use it

If you're carrying credit card debt, financing items like a watch or flights, juggling cash income from summer work, and want to actually hit a real savings target - FinanceOS gives you:

- A single dashboard showing **Net Worth, Total Debt, Cash, Coins, and Savings progress**
- A **debt payoff simulator** with Snowball and Avalanche strategies that tells you *exactly when you'll be debt-free*
- **Monthly budgets per category** that warn you before you overspend
- **Click-to-edit** every transaction and account - no remembering IDs
- **Automatic daily backups** so you can never lose data
- **One-click reset** so you can hand the app to a friend to test on their own data

Everything runs **locally on your PC**. The app binds to `127.0.0.1` only - your data never touches the network.

Installation (first time)

What you need

- Windows 10 or 11
- Python 3.11 or newer (download here - during install, **tick "Add Python to PATH"**)

Setup (run once)

The easy way - double-click `setup.bat`. It checks Python, creates the virtual environment, installs all dependencies. Re-running it is safe - it skips steps that are already done.

If `setup.bat` errors out, the message will tell you why (usually: Python not installed or not on PATH). Install Python 3.11+ from python.org/downloads with the **"Add Python to PATH"** box ticked.

Manual setup (only if `setup.bat` doesn't work for some reason): Open PowerShell in the FinanceOS folder and run:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If `Activate.ps1` is blocked, run this once: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`

Daily use - two ways to launch

Silent (recommended) - `launch.vbs`

Double-click `launch.vbs`. The browser opens at `http://127.0.0.1:8501` **without showing any terminal window**. The Streamlit process runs hidden in the background. This is the cleanest UX, especially for handing the app to non-technical users.

To **stop** the app later: double-click `stop.bat`. It finds whatever is listening on port 8501 and kills it.

Verbose (for debugging) - `run.bat`

Double-click `run.bat`. A terminal window appears with Streamlit's startup logs and any errors. The browser opens the same way. To close, close the terminal. Use this when something breaks and you need to see why.

Your first 5 minutes

1. **Add at least one account** (Accounts page -> Add)
2. **Record a transaction or two** (Transactions page -> Add)
3. **Set a budget** for at least one category (Budgets page)
4. **Set the minimum payment** for any debt accounts (Debts page)
5. **Open Dashboard** - you'll see your KPIs and charts come alive

That's it. Use it for a week and patterns emerge.

The pages, explained

Dashboard (home)

Live snapshot, updated every time you record something.

Top KPI row: - **Net Worth** - sum of every account balance - **Total Debt** - what you owe across credit cards, financing, overdraft, and friend loans you've taken - **Cash** - chequing + savings + cash bills (positive balances only) - **Coins** - physical coin holdings, tracked separately from bills

Savings Progress section shows how close you are to your yearly target (default $20,000 - editable in Settings).

Charts: - **Cash flow** - last 6 months, income (green) vs expenses (red) per month - **Spending by category** (this month) - donut chart - **Budget burn-rate** - progress bars for each budget you've set, with alerts at 80% used and over-budget - **Top 5 spending** - biggest expense subcategories this month - **Recent transactions** - last 10

Accounts

Add one entry per real-world account. Types supported:

Type	When to use
chequing	Daily bank account
savings	Bank savings account
credit	Credit cards (Visa, Mastercard, etc.) - uses Limit + Available input
cash	Physical bills in your wallet
coins	Physical coins, separate from bills. Each Coins account has an optional **denomination breakdown** - see "Coin breakdown" below.
overdraft	Overdraft balance (negative)
financing	Things you bought on payment plans (watch, flights, electronics)
investment	Brokerage accounts, ETFs
loan	Money you've lent to a friend OR borrowed from a friend

Special inputs:

Credit cards ask for the **limit** and **available funds now** instead of a negative balance. So if you have a $5,000 limit Visa and $3,500 is still available, you owe $1,500. Easier than typing `-1500.00`.

Loans ask for the **friend's name** and a checkbox "**TO ME** (I borrowed from them)". Unchecked = you lent (positive asset). Checked = you borrowed (negative debt). The account name composes itself: "Loan to John" or "Loan from Sarah".

Click a row in any of the three Accounts tables (Credit cards / Loans / Other accounts) to open its edit form below.

Transactions

Record every expense and income. Move money between accounts.

Add a transaction (top form): 1. Pick a **Category** (Food, Housing, Income, etc.) - dropdown 2. Pick a **Subcategory** - second dropdown updates based on category 3. Choose **Date** (defaults to today) 4. Pick **Account** (where the money moved) 5. Type **Amount** (positive number - the app applies the sign based on category) 6. Optional **Notes** 7. Click **Add Transaction**

Sign rule: Income -> +, everything else -> -. You never type a minus sign.

Transfer between accounts (second section): - Pay credit card from chequing - Move money to savings - Withdraw cash - Lend money to a friend - Repay a friend - Convert coins to cash

Both sides commit atomically - if anything fails, neither account changes.

All transactions table: - Filter by account / category - **Click any row** to edit it - Transfers show as "Transfer" - they can only be deleted (not edited) since both sides have to stay in sync

Budgets

Set monthly spending caps per category. Watch progress bars as you spend.

1. Pick **year and month** (defaults to current)
2. Optionally click **"Fill empty budgets from "** to copy forward
3. Type a dollar amount for each category you care about
4. Set a category to `0` to remove its budget
5. Click **Save Budgets**

Income isn't a category here - you don't cap income. **Transfers don't count** - paying off your Visa doesn't burn your Food budget. **Credit card charges DO count** - buying groceries with your Visa still adds to Food spending.

Color states: - Green bar: under budget - Yellow caption: 80%+ used - slow down - Red caption: over budget by $X

Debts

Plan your way out of debt with two proven strategies. Track real payments vs minimums.

Top KPIs: Total Debt | Total Minimum Payments | Interest This Month

Step 1 - Set minimum monthly payments For each debt, look at your statement and enter the minimum. For credit cards this is usually 2–3% of the balance.

Step 2 - Enter extra payment budget How much you can put toward debt **above** the minimums each month. Try $200 first, then experiment.

Step 3 - Compare strategies - **Snowball** (smallest balance first) - psychologically motivating - **Avalanche** (highest interest rate first) - mathematically optimal

For each, you see Months to debt-free, Debt-free date, Total interest paid. A recommendation banner tells you which saves more money.

Per-debt status table shows you: - What you owe - The interest rate - The minimum you should be paying monthly - **What you've actually paid this month** - Status vs minimum ((yes) over / (!) short / (no) not paid) - Months until that specific debt is gone - Estimated payoff date

Payment history chart shows the last 6 months for any debt you pick - green bars met the minimum, orange bars fell short, dashed red line is the minimum threshold.

To record an actual payment - the easy way (recommended):

In **Transactions -> Add a transaction**, pick: - **Category**: `Debt Payments` - **Subcategory**: `Credit Card` / `Loan` / `Line of Credit` - A "**Pay which debt?**" dropdown appears with matching debt accounts - Pick the debt, the source account (chequing), and the amount - Click **Pay Debt** - both sides commit atomically.

Behind the scenes this creates a proper transfer tagged as a debt payment, so the payment shows up in budgets AND in per-debt "Paid this month" simultaneously.

Subcategory -> eligible debt types: | Subcategory | Shows | |---|---| | Credit Card | `credit` accounts with money owed | | Loan | `loan` (friend loans you owe) + `financing` (watch, flights) | | Line of Credit | `overdraft` accounts |

Alternative - the manual way: Transactions -> Transfer between accounts -> From chequing -> To the debt account. This works the same but doesn't auto-tag the entry as a debt payment.

Don't record debt payments as regular expenses - money would leave your chequing without reducing any debt. (If you've done this by accident, click the bad row in the transactions table -> Delete -> re-record using the Debt Payments shortcut.)

■ Calculator

Three utility tabs in one page. **Nothing here writes to the database** - it's a quick-math scratchpad so you don't need to alt-tab away.

- **Calculator tab**: type an expression with `+ − × ÷` and parentheses. Decimal precision (no float drift - `0.1 + 0.2 = 0.3` exactly). Function calls and variables are blocked for safety.
- **Currency converter tab**: pick From / To currency, type an amount, see the conversion live. Uses rates from Settings -> Exchange rates (auto-updated 4 times daily).
- **Cash counter tab** (Canadian): enter how many of each bill and coin you have. Totals update live in three buckets:
- **Bills total** - sum of $100/$50/$20/$10/$5 bills × their counts
- **Coins total** - sum of $2/$1/25¢/10¢/5¢ coins × their counts
- **Grand total** - sum of both
- The two subtotals match the **Cash** and **Coins** account types naturally - useful before depositing or for monthly reconciliation.

Settings

- **Display currency** - pick from CAD, USD, EUR, EGP. Changes the symbol shown beside all amounts. **Switching also resets the yearly savings target** because the old number was denominated in the old currency. **Does not auto-convert your stored transaction amounts** - see "Multi-currency" and "Exchange rates" below.
- **Exchange rates** - store conversion rates between any pair of supported currencies. Set manually OR fetch the latest from the internet with one click. Powers the in-Settings currency converter. See "Exchange rates" section below.
- **Yearly savings target** - change the goal from the default 20,000 (in the active currency)
- **Backups** - create manual backups with optional labels, restore from any saved backup, see list of all backups
- **Archived accounts** - when a loan or financing debt is fully paid off, the app auto-archives it (see "Auto-archive" below). This section lists every archived account with its full closing summary and an **Unarchive** button if you ever need to restore one.
- **Reset all data** - type `RESET` to wipe everything. Optional checkbox "Also delete ALL backups" for a true privacy wipe with no recovery. Without the checkbox, an automatic pre-reset backup is created.
- **About** - public version and privacy summary (always visible).
- **Developer settings** - password-gated section showing the database file path, backup directory, and a "Change password" form. Default password is `0000` - change it once unlocked.

Multi-currency (USD / CAD / EUR / EGP)

The app supports four currencies - for **display** and for **per-account denomination**:

Code	Currency	Symbol
CAD	Canadian Dollar	$
USD	US Dollar	$
EUR	Euro	€
EGP	Egyptian Pound	E£

Pick the **display currency** in **Settings -> Display currency**. This sets what currency the KPI cards aggregate to (Net Worth, Total Debt, Cash Available, etc.). The symbol updates everywhere.

Per-account currency. Each account can be denominated in its **own** currency - independent of the display currency. So you can have a USD chequing account inside a CAD-defaulted system; the balance is stored as USD, and KPIs convert it to CAD via the latest stored exchange rate.

How it works: - When you add or edit an account (Accounts page), pick the **Currency** dropdown (CAD/USD/EUR/EGP) - The account's `Balance` field is in that account's native currency - Each account row in the Accounts tables shows a **Currency** column so you always know what it's denominated in - Top KPIs (Assets / Liabilities / Net Worth / Cash / Coins / Total Debt / etc.) convert every account to the display currency before summing - using rates from **Settings -> Exchange rates** (auto-updated 4× daily) - **If a rate is missing**, the account is skipped from the KPI total AND a warning lists which accounts couldn't be converted, with a one-click link to refresh rates

Exception: Coins accounts are **CAD-only** (the denomination grid uses Canadian coins).

Old caveat (still partly true): the exchange-rate conversion currently applies to **account-level** aggregates (KPIs). Transaction-level analytics - Cash Flow chart, Spending pie, Budget burn-rate - still treat all amounts as the same currency under the hood. For accurate multi-currency tracking in those views, keep each transaction on an account of its native currency, or run separate FinanceOS folders.

When you actually need two currencies (e.g. CAD income + EGP family transfers)

The cleanest approach is **two separate FinanceOS instances**:

1. Set up your primary instance with your main currency (e.g. CAD).
2. Copy the entire FinanceOS folder to another location (e.g. `FinanceOS-EGP/`).
3. Open the copy -> **Settings -> Reset all data** (creates a pre-reset backup of the duplicated data, which you can discard).
4. **Settings -> Display currency** -> pick EGP.
5. Now you have two independent ledgers - one per currency.

To run both simultaneously: edit the second instance's `run.bat` to use a different port (e.g. `--server.port 8502`). Then both apps are open in separate browser tabs.

Auto-archive (one-and-done debts)

When you make the final payment on a **loan** or **financing** account - bringing its balance to exactly $0 - the app automatically:

1. **Hides the account** from active views (Accounts page, Transactions dropdowns, Debts page, Dashboard)
2. **Appends a closing summary to the account's Notes** like this: `=== Archived 2026-05-17 === Account opened: 2026-03-01 (77 days) Total paid: $200.00 across 3 payments`
3. **Shows a celebration message**: [*] `<account name> fully paid off and archived!`
4. The archived account remains accessible in **Settings -> Archived accounts** with the full summary always visible - and you can **Unarchive** it any time.

Why not credit cards or overdraft? Those are revolving facilities you keep using. Paying a Visa to $0 just means it's healthy - you'll spend on it again. Auto-archive only fires for one-and-done debts.

Undoing a final payment un-archives automatically. If you delete the transfer that closed out a loan, the balance goes back to negative and the account reappears in active views - no orphan accounts left behind.

Common workflows

"I bought groceries with my credit card"

Transactions -> Add -> Category: Food / Subcategory: Groceries / Account: Visa / Amount: $85.50

The Visa balance gets more negative (more owed). The grocery amount counts toward your Food budget.

"I paid $200 toward my Visa from chequing"

Transactions -> **Add a transaction** -> Category: `Debt Payments` -> Subcategory: `Credit Card` -> Pay which debt: `Visa - TD` -> From: `TD Chequing` -> Amount: `200` -> **Pay Debt**

Chequing drops $200, Visa's "Owed" drops $200, "Available" goes up $200. Per-debt status table on Debts page reflects it instantly. Counts toward your Debt Payments budget if you set one.

"I got $50 in coins from my summer laundry job"

First make sure you have a Coins account (Accounts -> Add -> Type: coins).

Transactions -> Add -> Category: Income / Subcategory: Coins / Account: Coins / Amount: $50

Dashboard "Coins" KPI updates.

"I want to deposit my coins into chequing"

Transactions -> Transfer -> From: Coins -> To: TD Chequing -> Amount: $40

"I lent John $100"

First-time: Accounts -> Add -> Type: loan / Friend's name: John / Unchecked / Amount: 100. Done.

Subsequent lending: Transactions -> Transfer -> From: TD Chequing -> To: Loan to John -> Amount: ...

"John paid me back $50"

Transactions -> Transfer -> From: Loan to John -> To: TD Chequing -> Amount: $50

Loan account balance drops by $50. If this brings it to exactly $0, the account **auto-archives** with a closing summary in Notes (open date, days outstanding, total paid, number of payments) - see "Auto-archive" below.

"I want to see when I'll be debt-free"

Debts page -> Enter your extra monthly budget -> Look at both columns.

If Avalanche shows "Debt-free date: Aug 2026 - saves $217 in interest" - that's the math-optimal path.

"How much did I spend on coffee this month?"

Transactions page -> Filter by Category: Food -> Look for Coffee subcategory entries.

Or check the **Top 5 spending** chart on the Dashboard if it ranks high enough.

Tips for hitting your goals

Eliminating debt: - Enter the **real interest rates** on every debt - the simulator needs them - Use Avalanche if you want max interest savings, Snowball if you need motivation wins - Aim to pay **more than** the minimums every month - even $20 extra compounds - Record every payment via **Debt Payments** category in Transactions (or as a Transfer) so the per-debt history shows your discipline - Loans and financing **auto-archive** when fully paid off - watch for the [*] celebration message

Hitting $20k/year savings: - $20,000 ÷ 12 = **$1,667/month** required - Treat savings deposits as Transfers (chequing -> savings) - Set a Savings budget category for monthly target contributions - Watch the Savings Progress bar on Dashboard climb week by week

Staying within budget: - Set budgets in week 1 of the month, review weekly - Yellow warning at 80% used = slow down - Red over-budget = no fixing the past, but reset for next month

---

Coin breakdown (for Coins accounts)

Each Coins account can store an optional **denomination breakdown** - exact counts of toonies, loonies, quarters, dimes, and nickels. This is **purely informational** and never affects the Balance field; it's a tracking tool so you can see at a glance what's actually in a given coin jar.

Where it appears: - Accounts page -> **Coins accounts** table - each row shows a compact summary like *"4 toonies, 5 loonies, 4 quarters"* plus the dollar value those counts add up to ("Breakdown sum") - Click any row -> an **inline bar chart** appears above the edit form, showing counts per denomination with dollar values labeled on each bar - Edit form has a 2-column grid of counts (Toonie, Loonie, Quarter, Dime, Nickel) - bump the numbers, click Save

Why "Breakdown sum" exists alongside "Balance": The breakdown counts and the Balance are stored independently. The app shows both so you can reconcile manually if you want them to match (e.g. you just counted: 4 toonies + 5 loonies + 4 quarters = $14.00, balance is $14.00 - good). If they diverge, the app does NOT auto-correct either side.

Workflow: count physical coins (the Calculator -> Cash counter tab is great for this) -> open the matching Coins account -> edit the breakdown counts -> save. The chart updates on the next render.

Exchange rates

Stored conversion rates live in the `exchange_rates` table. Two ways to populate them:

Auto-update (default - set and forget)

Rates auto-fetch from open.er-api.com - a free service from exchangerate-api.com (no API key required) - on these triggers:

- **On app start** (if rates are stale or missing)
- **At 09:00, 13:00, 17:00, and 21:00 daily** (while the app is running, a background thread checks every 15 minutes and fetches when a scheduled slot has passed)

Settings -> Exchange rates shows the last-updated timestamp ("3 min ago", "5 h ago", etc.) and base currency. A **"Refresh rates now"** button forces an immediate fetch - useful if you can't wait for the next scheduled slot.

Manual override (for offline use)

If the API is unreachable (no internet, firewall, downtime), expand **"Manual override"** under Exchange rates and set a specific rate yourself. The inverse is auto-derived (save CAD->EGP and EGP->CAD is computed for you).

Why this source?

- Free, no API key needed (so testers don't need to sign up for anything)
- Updates daily
- Supports CAD, USD, EUR, EGP (and ~160 others)
- Industry-standard exchangerate-api.com is behind it

Trade-offs vs. government sources:

- Bank of Canada's Valet API is government-backed but CAD-only (no EGP↔EUR, etc.)
- European Central Bank rates are official but don't include EGP
- exchangerate-api.com aggregates from multiple banks - practical for personal use, not for financial-audit-grade reporting

Currency converter widget

Below the rates list there's a quick converter - type any amount, pick From and To, and you'll see the converted value instantly. Uses whatever rates you have stored.

What this DOESN'T do (yet)

The app does NOT automatically convert your historical transaction amounts when you change the display currency. If you have a chequing balance of `2,000` (originally CAD) and switch to EGP, the dashboard will display "E£2,000.00" - same number, just relabeled. The exchange rate feature is for one-off calculations in Settings, not retroactive conversion of your ledger.

For tracking multiple currencies properly (e.g. separate CAD account + EGP account in the same view), the cleanest path is still **separate FinanceOS folders per currency** - covered in the Multi-currency section above.

Developer settings (password lock)

The bottom of the Settings page has a section labelled **Developer settings ■**. It hides the **database file path** and **backup directory path** behind a password so a tester who's using your app for a week can't see

where your private data lives on disk.

- **Default password:** `0000`
- **To unlock:** type the password and click Unlock - info is shown until you close the app
- **To change the password:** unlock first, then use the "Change developer password" form (requires current password + new password + confirmation)
- **After Reset all data:** the password resets to the default `0000` because all settings are wiped

This is a soft barrier - not encryption. Anyone with file-system access to the database can still see the plain password. The point is to hide implementation details from casual users of the app, not to harden it against attackers (Windows account access is the real boundary).

True privacy wipe before handing off

When you give the app to a tester or sell/donate the device, the **Reset all data** button always creates a "pre-reset" backup so you can restore later - but those backup files still contain your data. For a **clean handoff with no traces**:

1. Settings -> scroll to **Reset all data**
2. Type `RESET`
3. Tick **"Also delete ALL backups (true privacy wipe - NO recovery possible)"**
4. Click **(!) Wipe all data**

Result: every transaction, account, debt, budget, archived item, setting, and backup file is gone. The DB schema is recreated empty. Nothing on disk can be used to recover your data.

If you might want your own data back later, do a normal reset (uncheck the box) - the pre-reset backup will be waiting for you.

Privacy & data

- **Where your data lives:** `<project_folder>/data/finance.db` - a single SQLite file, **always inside this exact folder.** If you copy the FinanceOS folder to a new location (e.g. another PC, USB stick, second copy on the same machine), the copy has its own independent `data/finance.db`. **Two copies of the app never share data unless you manually copy the `data/` folder between them.**
- **Backups:** `<project_folder>/data/backups/finance_YYYY-MM-DD_HH-MM-SS.db` - one created automatically per day, kept for 14 days
- **Network:** the app binds to `127.0.0.1:8501` only. No other device on your network can reach it.
- **No login:** the app trusts that whoever is at your Windows session is you
- **To completely wipe:** Settings -> Reset all data (with auto-backup), OR delete the entire `data/` folder for a totally clean slate

Moving to a different computer

Copy the entire FinanceOS folder (or just the `data/` folder if FinanceOS is already installed on the target). All your data + backups travel with the file.

Sharing with a tester

1. **Settings -> Reset all data** (this creates a pre-reset backup so YOUR data is safe)
2. Hand them the folder (or have them install fresh)
3. They use it like it's theirs - separate database, no overlap with yours
4. When you get the folder back, **Settings -> Restore** -> pick your pre-reset backup. Your data is back instantly.

Troubleshooting

"`Activate.ps1` cannot be loaded" error Run this once: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`

Browser didn't open automatically Open it manually and go to `http://127.0.0.1:8501`

"Already in use" port error Another Streamlit instance is running. Close the other terminal window first.

App shows old data after restore Hit **R** in the browser or refresh the tab. Streamlit caches some elements but a refresh always re-queries the DB.

Per-debt "Paid this month" stays at $0 even though I paid You probably recorded the payment as a regular **expense** instead of a **Transfer**. Edit the transaction and re-create as a Transfer (chequing -> debt account).

I want a different savings target Settings -> Yearly savings target -> enter your number -> Save. Dashboard updates immediately.

The app crashes / errors Restart `run.bat`. Your data is safe - the auto-backup runs every time you open the app on a new day.

Known limitations (v1)

- **CSV bank import** - not in v1. Record transactions manually for now.
- **Multi-currency auto-conversion** - display currency is switchable (USD/CAD/EUR/EGP) AND exchange rates can be stored and used for the in-Settings converter widget. But stored transaction amounts are NOT auto-converted when you change display currency - they're just re-labeled with the new symbol. For true multi-currency tracking use separate FinanceOS folders per currency.
- **Mobile app** - desktop only. Runs in your browser locally on the PC.
- **Investment tracking** - basic only (account-level balance). No share-price tracking.

- **Recurring transactions** - not automated. Re-enter monthly bills manually.
- **AI insights / spending anomaly alerts** - deferred. The charts + budget burn-rate already surface most red flags.
- **Password lock** - not in v1. The app trusts your Windows session.

Categories cheat sheet

All categories are fixed dropdowns (no free typing) so analytics stay clean:

Category	Subcategories
Housing	Rent, Utilities, Internet, Maintenance
Food	Groceries, Restaurants, Coffee, Takeout
Transport	Gas, Transit, Parking, Car Insurance, Car Maintenance
Health	Pharmacy, Dental, Insurance, Gym
Personal	Clothing, Haircut, Laundry, Hobbies
Subscriptions	Streaming, Software, Phone
Education	Tuition Fees, Books, Supplies, Student Fees
Debt Payments	Credit Card, Loan, Line of Credit
Savings	Emergency Fund, Goal Contribution
Income	Salary, Cash Income, Coins, Bonus, Refund, Other
Other	Gift, Travel, Misc

Note: "Coins" is its own income subcategory but **rolls up to "Cash Income"** in summary analytics - you can track coins explicitly while still having a single "physical money received" total.

Feedback

If you're a tester: keep a notepad open while you use the app for a week. Note: - Anything that confused you - Anything you tried to do that the app wouldn't let you - Any features missing that you needed - Any

numbers that looked wrong

This list is what shapes v2.